Real-World PBTs as Lean Challenges

Anonymous Author(s)
Affiliation
Address
email

Abstract

We present a benchmark for evaluating AI models on real-world formal software verification tasks. We first scrape ≈20k property-based tests (PBTs) from permissively licensed real-world Python repositories, then automatically translate them into Lean 4 specifications with `sorry` placeholders. Translating PBTs into formal specifications is challenging: it requires modeling Python semantics in Lean, bridging the gap between input/output testing and structural proofs, and handling the inherent difficulties of dependently-typed programming. We describe a three-agent LLM pipeline for transpiling PBT suites into Lean specifications, evaluate coverage and quality metrics, and provide baselines for proof completion using several automated and model-based approaches. Our benchmark aims to drive progress on the underexplored problem of AI-assisted formal verification of real-world software, which we claim is of increasing import as AI begins to produce an unprecedented percentage of the world's code.

1 Introduction

Several AI safety proposals are premised on AI-driven formal verification (FV); this is a core hypothesis of the ARIA Safeguarding AI program and the related Guaranteed Safe AI agenda Dalrymple et al. [2024]. If FV is to play this role, it must scale to the complexities of real-world code, but today we have little evidence that such capabilities are possible. Relevant benchmarks are mostly small datasets crafted by hand Loughridge et al. [2024], larger datasets extracted from expert-driven verification projects Chakraborty et al. [2025], or are focused on advanced mathematics rather than engineering Glazer et al. [2024]. None of these are likely to be representative of the formal verification tasks required to verify real-world AI-generated software at scale.

We generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a 'close sibling' technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(lst)` always returns a sorted permutation of its input `lst`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams Dodds [2022]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s, any unproven theorem is automatically treated as a PBT (and tested against randomly generated inputs) until the author manages to supply a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard. We build on this intuition. First, we scrape ≈20k permissively-licensed PBTs written in Python's Hypothesis framework MacIver et al. [2019]. Each PBT in our dataset can be seen as a small theorem statement (or *ansatz*) about a piece of Python code, which the author would like to prove or, failing that, at least test. To make it possible for an AI to

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

prove or refute these theorems, we then translate both code and PBTs to the theorem prover Lean, which is becoming the de-facto standard in AI formal verification. This approach was pioneered by Dougherty and Mehta on the FVAPPS verification benchmark Dougherty and Mehta [2025]. In FVAPPS, source programs were taken from the APPS benchmark by Hendrycks et al Hendrycks et al. [2021], translated to PBTs, and then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

We use AI-driven translation to generate Lean programs and theorems at scale. LLMs are quite good at program translation, and FVAPPS has shown that this approach is practical for constructing Lean theorems. Although in some cases the translations may be imperfect, lessening the importance of any proof or refutation of the resultant theorem from the perspective of the author of the originating PBT, it is nevertheless the case that we produce high-quality and realistic FV challenges for AI benchmarking. The result is tens of thousands of verification challenges, each consisting of (1) a program and (2) a desired specification, both written in Lean. Unlike existing FV benchmarks, each problem corresponds to properties of real-world software written by engineers with no guaranteed formal verification experience.

TODO: Summary of results.

2 Task Definition

Each problem in the FVSpec benchmark consists of three components: (1) the original Python property-based test, (2) a Lean theorem corresponding to the PBT, with a sorry placeholder where the proof should go, and (3) a Lean implementation of the functions referenced in the theorem. The AI's task is to complete the proof by replacing sorry with a valid Lean proof term. Each problem also includes a URL pointing to the original source code and license information.

2.1 Running Example

Consider a Hypothesis test asserting the Pythagorean trigonometric identity $\sin^2(x) + \cos^2(x) = 1$, for a trigonometric library `trg`:

```
from hypothesis import given, strategies

@given(x=strategies.floats(
    min_value=-sys.float_info.max / 4,
    max_value=sys.float_info.max / 4,
    allow_infinity=False))
def test_property(x):
    assert trg.sine(x) ** 2 + trg.cosine(x) ** 2 == pytest.approx(1)
```

Our pipeline translates this into a Lean implementation:

```
namespace Fvspec.Impl

def to_radians (degrees : Float) : Float :=
  degrees * Float.pi / 180.0

def sine (theta : Float) : Float :=
  Float.sin (to_radians theta)

end Fvspec.Impl
```

And a corresponding Lean specification:

```
import Fvspec.Impl
open Fvspec.Impl

def cosine (theta : Float) : Float := sorry

theorem sine_cosine_pythagorean (x : Float) :
  (sine x) * (sine x) + (cosine x) * (cosine x) = 1 := sorry
```

The AI must fill in both `sorry` placeholders with valid Lean proof terms.

TODO– Explain how we do not faithfully translate the original `trg` library into Lean, with all dependencies and so forth, rather, we utilize Lean's built-in `sine` and `cosine` functions. This is representative of our general attitude which is that our primary goal is to produce equally interesting, realistic problems to benchmark AI on, while actually proving properties about this real-world software to help these specific code authors is just a secondary goal.

2.2 PBT vs. Proof

In PBT, a property is checked by randomly generating inputs and testing them—the process is push-button but provides only statistical confidence. In FV, the property is proved using computer-aided symbolic logic, providing near-perfect confidence but requiring proof engineering.

Translating a PBT into a formal specification thus involves a shift from input/output testing to proofs over the structure of the program. Our pipeline filters nonsensical theorems, but any discrepancies that remain will not affect the usefulness of the dataset. Because the Lean theorem prover provides a 'perfect' oracle, every theorem we generate is a useful challenge problem, even if it differs from the original PBT. In fact, some PBTs in our input dataset will be naturally false due to undetected edge cases.

TODO- Explain what we mean by nonsensical theorems.

TODO- Explain to what degree Lean can be viewed as a perfect oracle.

2.3 Challenges

Formal verification is unusually difficult to benchmark for two reasons. First, just as with advanced mathematics, formal verification tasks are highly varied, and constructing interesting problems requires considerable expertise. FV tasks also vary in difficulty—they can be as shallow as checking the type-safety of simple functions, or as deep as proving an unsolved mathematical theorem such as the Collatz Conjecture. Our benchmark avoids the need to select and hand-build challenge problems by translating naturally occurring PBT problems written by real-world engineers.

Second, formal verification is little-used outside academia. As a result, there are few natural datasets, and those that exist are fragmented between multiple FV tools (e.g. SAW/Cryptol for cryptographic systems, ACL2/Coq for traditional software verification, Lean for mathematics, SPIN/TLA+ for protocol analysis)—or are privately held, closely guarded corporate IP (such as the FV work done by companies like Intel or Synopsys). Public FV examples are often pedagogical exercises for which the proof exists in many different forms in the training set. Our benchmark solves this problem by translating from a 'close sibling' domain, PBT, where many more datapoints exist outside of pedagogy or academic research. As a result, most theorems in our benchmark will have never been verified by anyone.

TODO- A reviewer will ask us to substantiate this claim.

Beyond these structural challenges, AI-driven translation at scale introduces practical difficulties. Each failed pipeline run is expensive—a single sample may require multiple LLM calls with iterative repair, and a bug in the pipeline infrastructure can invalidate an entire batch. Quality improvements must therefore be targeted: we identify specific failure modes in the output (e.g. type restrictions, missing dependencies, trivial stub theorems) and address them incrementally rather than re-running the full dataset. We initially considered extending the benchmark to TypeScript PBTs, but found that the PBT ecosystem in TypeScript is far less mature, and the additional coverage would not justify the engineering effort.

3 Pipeline

Our generation pipeline is modelled on FVAPPS Dougherty and Mehta [2025], but starts from real-world Hypothesis property-based tests (PBTs) rather than synthetic programming puzzles. For each sample, we (i) recover the Python function under test (FUT) and dependencies, (ii) translate the code to a computable Lean implementation, (iii) translate the PBTs into Lean theorems, and (iv) score and filter the resulting artifacts (Figure 1).

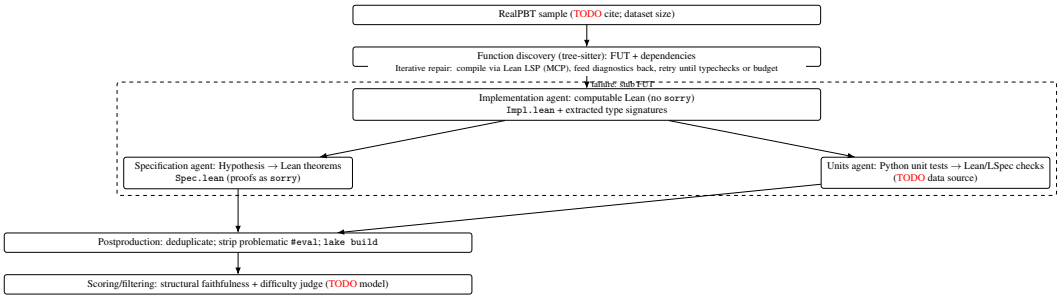

Figure 1: FVSpec pipeline: discover FUT+deps, translate to computable Lean, generate specs, repair via Lean LSP, then postprocess and score.

3.1 Function discovery

For each PBT, we use tree-sitter to extract the Python source code of the FUT and any locally-defined dependencies. This function discovery step succeeds for TODO % of samples; when it fails, we generate a stub FUT with generic type parameters to avoid producing trivial theorems.

3.2 LLM-based transpilation

We translate each sample using three agents.

Implementation agent The implementation agent runs first, translating the FUT and its dependencies into fully computable Lean definitions. No `sorry` placeholders are permitted, so the output must typecheck and evaluate. Dependencies are formalized separately and then merged with deduplication into a single `Impl.lean` file. On success, we additionally extract Lean type signatures for all formalized functions and provide them as context to downstream agents. If the agent exhausts its retry budget without producing typechecking code, downstream stages still run, but without a verified implementation to build on.

Specification and units agents Next, the specification agent and units agent run in parallel. The specification agent translates Hypothesis tests into Lean theorems that import the formalized implementation; proofs are left as `sorry`. The units agent translates any available Python unit tests into Lean/LSpec checks. The units output is included as auxiliary validation, but does not gate inclusion of the specification (TODO confirm policy).

3.3 Iterative repair against Lean

All agents operate in an iterative repair loop. At each step, we check the generated Lean code against the compiler via the Lean LSP (accessed through an MCP tool server). If the code does not typecheck, we feed the compiler diagnostics back to the agent and prompt for a revised translation. This loop continues until the code typechecks or a retry budget is exhausted. We validate final compilation with a full `lake build`.

3.4 Cleanup and quality evaluation

We apply postproduction passes to remove known failure modes (e.g., stray `#eval` fragments that can pass in isolation but break a project build) and to merge/deduplicate translated dependencies across files.

The resulting specification/implementation pairs are then scored and filtered. We compute *structural faithfulness* programmatically by comparing the generated Lean artifacts to the source PBTs. The metric decomposes into sub-scores for parameter coverage, type correspondence, strategy coverage, assertion coverage, and dependency coverage, producing an overall score (TODO define weights; add exact formula). We also assign a *difficulty* rating using an LLM judge (TODO specify model and prompt), intended to approximate the proof effort required on a 1–10 scale.

4 Results

TODO Draft placeholders only: experimental setup, coverage, quality, baselines, difficulty stratification (with histogram of Haiku-assessed difficulty grades across the benchmark tasks), human baselining (???).

5 Threats to Validity

We note that the theorems we construct may be false—this is desirable because the AI has the opportunity to contradict the theorem (and perhaps identify a counter-example). Because the Lean theorem prover is a so-called foundational tool, once a theorem is verified or contradicted, we have near-perfect confidence the answer is correct. However, because our specifications are translated rather than hand-written, some may not faithfully represent the original PBT. Our structural faithfulness metric provides an objective measure of translation quality, but it is a heuristic: a high score does not guarantee semantic equivalence, and a low score does not necessarily indicate a useless problem.

Our benchmark does not include human baselines. We expect the dataset to exhibit a wide range of difficulty, including many problems that are trivial for current AIs, and many harder problems, including some insurmountable even to world-expert human teams. Without human expert performance data, we cannot directly calibrate this difficulty distribution.

The pipeline's reliance on LLM-driven translation means that benchmark quality is bounded by the capabilities of the translation model. Samples where function discovery failed (8%) produce stub-based theorems that may be less representative. We mitigate this through postproduction filtering and the structural faithfulness metric, but some lower-quality samples may remain.

6 Related Work

Our work extends FVAPPS by Dougherty and Mehta Dougherty and Mehta [2025], which pioneered AI-driven translation of coding problems into Lean theorems. In FVAPPS, source programs were taken from the APPS benchmark Hendrycks et al. [2021], translated to PBTs, then lifted to theorems. We apply the same process at scale to PBTs found in the wild, rather than synthetic coding challenges.

Lean verification benchmarks. A growing number of benchmarks target Lean specifically. miniCodeProps Brandfonbrener et al. [2024] provides 177 program specifications translated from TIP (Tons of Inductive Programs), finding that current LLM provers fail on medium and hard properties. CLEVER Materzok et al. [2025] offers 161 curated problems sourced from HumanEval, testing end-to-end verified code generation. Verina Wu et al. [2025b] covers all three foundational tasks—code generation, specification generation, and proof generation—across 189 manually curated coding tasks. VeriBench Stechly et al. [2025] translates Python code with documentation into Lean implementations, specifications, and unit tests (113 tasks). The Vericoding Benchmark Bursuc et al. [2025] supports Lean, Rust/Verus, and Dafny, reporting success rates from 27% (Lean) to 82% (Dafny). VeriEquivBench Ma et al. [2025] uses LeetCode transformations to evaluate verifiable agents on novel data without contamination. These benchmarks are valuable but share a common limitation: their specifications derive from synthetic coding problems or pedagogical exercises, not from properties written by practicing engineers.

Other verification tool benchmarks. DafnyBench Loughridge et al. [2024] provides Dafny verification tasks. VerusBench Chen et al. [2024] collects 150 proof tasks spanning Dafny, SV-COMP, and Verus. SV-COMP Beyer [2025] is the standard competition for software verifiers, with over 33,000 C tasks, but targets automated verification rather than interactive theorem proving. InvBench Wu et al. [2025a] focuses specifically on loop invariant synthesis. Chakraborty et al. Chakraborty et al. [2025] explore neural synthesis for SMT-assisted proof-oriented programming in F*.

Interactive theorem proving benchmarks. LeanDojo Benchmark 4 Yang et al. [2024] provides 122,517 theorems from Mathlib4. CoqStoq Thompson et al. [2025] collects 196,929 theorems from 2,226 GitHub Coq repositories. These are primarily mathematical rather than software-oriented, and their theorems exist in the training data of most frontier models.

Mathematics benchmarks. FrontierMath Glazer et al. [2024] evaluates advanced mathematical reasoning. While formal verification and mathematics share proof infrastructure, the skills required differ substantially: software verification demands modelling of program semantics, library APIs, and computational behavior, rather than abstract mathematical reasoning.

FVSpec differs from existing benchmarks in three ways. First, our specifications originate from real-world property-based tests written by practicing engineers, not synthetic problems or pedagogical exercises. Second, the source PBTs were never formally verified, so most theorems in our benchmark will not appear in any model's training data. Third, we operate at a larger scale than existing Lean verification benchmarks, with thousands rather than hundreds of problems. Our work is motivated by the broader context of AI safety proposals premised on AI-driven formal verification, including the Guaranteed Safe AI agenda Dalrymple et al. [2024].

7 Conclusion / Future Work

Several extensions are possible:

TODO: Add examples from other languages with significant usage of PBT, e.g. Haskell/QuickCheck, C++/RapidCheck. Translate our benchmark to other theorem proving systems, e.g. Coq, Isabelle, F*, etc. More advanced website features, such as prediction markets about when the benchmark will be saturated, and a pipeline for submitting code to the leaderboard which we then persist into a training or SFT dataset. Our benchmark will also be useful for developing new techniques in program repair and synthesis. If a property is disproven, this creates a new interesting challenge, namely to synthesize a patch to the original function such that the theorem now holds. We hypothesize that many such bugs will be particularly subtle and interesting edge cases since, by definition, in most cases they were not detected by the original developer's PBT harness.

References

D. Beyer. State of the art in software verification and witness validation: SV-COMP 2025. In *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2025.

D. Brandfonbrener, L. Beurerkellner, D. Romero, N. Amin, and M. Vechev. miniCodeProps: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.

S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, M. Zhao, M. Tan, and M. Tegmark. A benchmark for vericoding: formally verified program synthesis, 2025. URL `https://arxiv.org/abs/2509.22908`.

S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.

H. Chen et al. VerusBench: A benchmark for automated verification of Rust with Verus. *arXiv preprint arXiv:2409.13082*, 2024.

D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.

M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022. doi: 10.1109/MSEC.2022.3153035.

Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.

E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järviniemi, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla,

and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.

D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.

X. Ma et al. VeriEquivBench: Evaluating verifiable agents on novel data without contamination. *arXiv preprint arXiv:2510.06296*, 2025.

D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.

M. Materzok, W. Szulc, M. Tabiś, J. Kozak, and B. Piotrowski. CLEVER: A benchmark for closed-loop end-to-end formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.

K. Stechly et al. VeriBench: Benchmarking end-to-end formal verification. In *AI4Math Workshop at ICML*, 2025. URL `https://openreview.net/forum?id=rWkGFmnSNl`.

K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner, and E. First. Rango: Adaptive retrieval-augmented proving for automated software verification, 2025. URL `https://arxiv.org/abs/2412.14063`.

T.-Y. Wu et al. InvBench: A benchmark for loop invariant synthesis. *arXiv preprint arXiv:2509.21629*, 2025a.

W.-D. Wu, K. Wang, R. Mangal, and I. Dillig. Verina: Benchmarking formally verified code generation. *arXiv preprint arXiv:2505.23135*, 2025b.

K. Yang, A. Swope, A. Gu, R. Chalapathy, P. Song, S. Lember, A. Thomson, and A. Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.